

INFORME DE PROYECTO

SmartFix

Sistema de Gestion de Reparaciones

Arquitectura de Microservicios con Spring Boot

Gateway · BFF · 11 microservicios de negocio · Observabilidad (GlitchTip)

Integrantes

Jeferson Aquino

Karl Obrequé

Jorge Leal

Ingeniería de Software
2026

Índice

[Índice](#)

[1. Introduccion](#)

[2. Desarrollo](#)

[2.1 Requerimientos Funcionales](#)

[Modulo de Autenticacion y Usuarios \(ms-cliente\)](#)

[Modulo de Reparaciones \(ms-reparacion\)](#)

[Nueve microservicios de soporte al negocio](#)

[Modulo BFF y Gateway](#)

[2.2 Requerimientos No Funcionales](#)

[2.3 Casos de Uso Principales](#)

[CU-01: Registro y Login de Usuario](#)

[CU-02: Gestion de Clientes](#)

[CU-03: Creacion de Reparacion con Verificacion de Cliente](#)

[CU-04: Consulta del Dashboard \(BFF\)](#)

[CU-05: Acceso sin Credenciales \(403 Forbidden\)](#)

[CU-06: Acceso a traves del Gateway](#)

[CU-07: Calculo de Cotizacion](#)

[CU-08: Registro y Consulta de Auditoria](#)

[CU-09: Reporte de un Error a GlitchTip](#)

[2.4 Explicacion de los Microservicios](#)

[gateway \(puerto 8080\)](#)

[bff-web \(puerto 8083\)](#)

[ms-cliente \(puerto 8081\) y ms-reparacion \(puerto 8082\)](#)

[Microservicios sin base de datos: ms-notificacion, ms-cotizacion, ms-garantia, ms-conversor y ms-diagnostico](#)

[Microservicios con base de datos propia: ms-inventario, ms-factura, ms-encuesta y ms-auditoria](#)

[2.5 Diagrama C2 — Arquitectura de Contenedores](#)

[2.6 Diagrama C3 — Componentes](#)

[2.7 Cobertura de Pruebas](#)

[3. Conclusion](#)

1. Introduccion

SmartFix es un sistema de gestion de reparaciones tecnicas construido sobre una arquitectura de microservicios en Spring Boot. Su proposito central es ofrecer una plataforma robusta que permita a talleres tecnicos o centros de servicio registrar clientes, administrar el ciclo completo de una reparacion, calcular cotizaciones y garantias, controlar inventario de repuestos, emitir facturas, medir la satisfaccion del cliente y auditar la actividad del sistema, todo ello autenticado mediante tokens JWT.

El sistema se desarrollo en tres etapas incrementales. En la primera (C1) se construyeron los dos microservicios de negocio originales, ms-cliente y ms-reparacion. En la segunda (C2) se incorporo el Backend For Frontend (bff-web), que orquesta ambos servicios para simplificar la integracion con el cliente web o movil, y se reforzo la seguridad de las llamadas internas. En la tercera y ultima etapa (C3) se agrego un API Gateway como unico punto de entrada publico del sistema, nueve microservicios adicionales que complementan el negocio (notificaciones, cotizaciones, garantias, conversion de moneda, diagnostico, inventario, facturacion, encuestas y auditoria) y una capa de observabilidad centralizada con GlitchTip.

El problema de negocio que resuelve SmartFix es la dispersion y la falta de trazabilidad que existe en la gestion manual de reparaciones: sin un sistema centralizado, los tecnicos pierden el estado de cada equipo, los clientes no reciben actualizaciones oportunas, la administracion carece de visibilidad sobre la carga de trabajo y no existe un registro auditable de lo que ocurre en cada servicio. SmartFix centraliza esa informacion en trece componentes independientes, mantenibles y seguros, cada uno responsable de una unica capacidad de negocio.

El presente informe documenta la arquitectura tecnica final del sistema, los requerimientos funcionales y no funcionales, los casos de uso principales y la estructura completa de microservicios que componen la solucion, siguiendo el modelo C4 para la representacion de diagramas.

2. Desarrollo

2.1 Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades concretas que el sistema provee a sus usuarios, agrupadas por microservicio.

Modulo de Autenticacion y Usuarios (ms-cliente)

- **RF-01:** El sistema debe permitir registrar un nuevo usuario mediante correo electronico y contrasena, retornando un token JWT al completar el registro exitosamente.
- **RF-02:** El sistema debe autenticar a un usuario existente mediante sus credenciales (login), retornando un token JWT valido por 1 hora.
- **RF-03:** El sistema debe permitir crear un cliente con datos personales (RUT unico, nombre, telefono, correo). El RUT duplicado debe ser rechazado con error 409.
- **RF-04:** El sistema debe permitir listar, obtener, actualizar y eliminar clientes registrados (CRUD completo).

Modulo de Reparaciones (ms-reparacion)

- **RF-05:** El sistema debe permitir crear una reparacion, verificando previamente que el cliente (RUT) exista en ms-cliente mediante un token JWT interno.
- **RF-06:** El sistema debe asignar el estado inicial RECIBIDO a toda reparacion creada y permitir su transicion a EN_PROCESO y LISTO.
- **RF-07:** El sistema debe permitir listar todas las reparaciones y filtrarlas por RUT de cliente.

Nueve microservicios de soporte al negocio

- **RF-08:** ms-notificacion debe simular el envio de una notificacion por EMAIL o SMS a partir de un destinatario, canal y asunto (POST /api/notificaciones/enviar).
- **RF-09:** ms-cotizacion debe calcular el presupuesto estimado de una reparacion segun tipo de dispositivo, tipo de falla y nivel de urgencia (POST /api/cotizaciones/calcular).
- **RF-10:** ms-garantia debe calcular la fecha de vencimiento de la garantia de una reparacion segun su tipo (POST /api/garantias/calcular).
- **RF-11:** ms-conversor debe convertir un monto en pesos chilenos (CLP) a dolar (USD), euro (EUR) o peso argentino (ARS) (GET /api/conversor/convertir).
- **RF-12:** ms-diagnostico debe sugerir una prioridad de atencion y un tiempo estimado de reparacion a partir de los sintomas reportados (POST /api/diagnostico/analizar).
- **RF-13:** ms-inventario debe exponer un CRUD completo de repuestos por SKU y permitir ajustar el stock disponible (PATCH /api/repuestos/{sku}/stock).
- **RF-14:** ms-factura debe permitir emitir una factura, consultarla por folio o por RUT de cliente, y anularla (PATCH /api/facturas/{folio}/anular).
- **RF-15:** ms-encuesta debe registrar encuestas de satisfaccion asociadas a una reparacion y calcular el promedio general de satisfaccion (GET /api/encuestas/promedio).
- **RF-16:** ms-auditoria debe permitir que cualquier microservicio registre un evento (POST /api/auditoria/registro) y consultar el historial por servicio de origen.

Modulo BFF y Gateway

- **RF-17:** El BFF debe exponer un endpoint de login unificado que delegue la autentificacion en ms-cliente.
- **RF-18:** El BFF debe exponer un endpoint de dashboard que combine datos del cliente y sus reparaciones en una sola respuesta JSON, usando llamadas reactivas y en paralelo a ambos microservicios.
- **RF-19:** El BFF debe exponer los once microservicios de negocio bajo el prefijo /bff/**, retornando 503 con el nombre del servicio afectado si este no responde.
- **RF-20:** El Gateway debe ser el unico punto de entrada publico del sistema, reenviando todo el trafico de /bff/** y el Swagger del BFF hacia bff-web.

2.2 Requerimientos No Funcionales

- **RNF-01:** Seguridad. Todos los endpoints de negocio estan protegidos con JWT (HS256). Las unicas rutas publicas son /api/auth/**, /bff/auth/** y la documentacion Swagger.
- **RNF-02:** Autenticacion interna segura. La comunicacion entre ms-reparacion y ms-cliente se realiza mediante un token JWT interno de rol INTERNAL_SERVICE y duracion de 60 segundos, eliminando rutas publicas inseguras.
- **RNF-03:** Bases de datos independientes. Cada microservicio con persistencia posee su propia base de datos PostgreSQL (Database per Service); ninguno comparte tablas con otro.
- **RNF-04:** Migraciones versionadas. Flyway gestiona el esquema de cada base de datos de forma reproducible.
- **RNF-05:** Manejo centralizado de errores. Todos los microservicios implementan @RestControllerAdvice para estandarizar las respuestas de error con timestamp, status, error y mensaje.
- **RNF-06:** Documentacion de API. Cada servicio expone Swagger UI (springdoc-openapi, OpenAPI 3.0); el Swagger del BFF agrupa a los once microservicios de negocio.
- **RNF-07:** Contenerizacion. El proyecto se despliega con Docker y Docker Compose, incluyendo healthchecks para las seis bases de datos.
- **RNF-08:** Pruebas automatizadas. El proyecto incluye mas de 60 pruebas unitarias y de integracion (JUnit 5, Mockito, H2) repartidas entre los trece componentes, con cobertura minima de 40% (JaCoCo) en los microservicios de negocio.
- **RNF-09:** Punto de entrada unico. Ningun cliente externo habla directo con un microservicio: todo el trafico entra por el Gateway (puerto 8080) y se reenvia al BFF, unico componente autorizado a llamar a los once microservicios de negocio.
- **RNF-10:** Observabilidad. Los errores de nivel ERROR de cualquier microservicio se reportan automaticamente a GlitchTip (autoalojado, compatible con el SDK de Sentry) ademas de mostrarse por consola (Logback).
- **RNF-11:**Codigo en ingles. Todas las clases, variables, metodos y tablas estan escritas en ingles.

2.3 Casos de Uso Principales

CU-01: Registro y Login de Usuario

El cliente HTTP realiza POST /api/auth/register con username y password. El AuthController delega en AuthService, que valida la ausencia de duplicados, cifra la contraseña y almacena el usuario, retornando un AuthResponseDTO con el token JWT. Para el login, el flujo es analogo mediante POST /api/auth/login.

CU-02: Gestion de Clientes

Con el token JWT en el header Authorization, el usuario llama a POST /api/customers. El ClienteController delega en ClientServiceImpl, que valida el RUT unico y persiste el registro. En caso de RUT duplicado, el GlobalExceptionHandler retorna un 409 Conflict estandarizado.

CU-03: Creacion de Reparacion con Verificacion de Cliente

El usuario llama a POST /api/reparaciones con rutCliente, modelo y descripcion. Antes de persistir, ReparacionServiceImpl invoca ClienteVerificacionService, que genera un token JWT interno (60 s, rol INTERNAL_SERVICE) y consulta a ms-cliente. Si el cliente no existe, se retorna 404; si existe, se guarda la reparacion con estado RECIBIDO.

CU-04: Consulta del Dashboard (BFF)

El frontend realiza GET /bff/dashboard/{rut}. BffServiceImpl usa WebClient reactivo y lanza en paralelo (Mono.zip) dos llamadas: una a ms-cliente y otra a ms-reparacion. Los resultados se combinan en un DashboardResponseDTO y se retornan en una sola respuesta.

CU-05: Acceso sin Credenciales (403 Forbidden)

Si el cliente intenta consumir cualquier endpoint protegido sin enviar el header Authorization, el JwtFilter (presente en cada microservicio) intercepta la peticion y retorna 403 Forbidden, sin llegar a la capa de negocio.

CU-06: Acceso a traves del Gateway

Un cliente externo realiza cualquier peticion de negocio contra http://localhost:8080/bff/**. El Gateway (Spring Cloud Gateway, variante WebMVC) reenvia la peticion tal cual, incluyendo el header Authorization, hacia bff-web en el puerto interno 8083. El Gateway no valida ni modifica el token: esa responsabilidad es exclusiva del BFF.

CU-07: Calculo de Cotizacion

El usuario llama a POST /bff/cotizaciones/calcular indicando tipo de dispositivo, tipo de falla y urgencia. El BFF reenvia la peticion a ms-cotizacion mediante RestClient, que aplica sus reglas de negocio y retorna el presupuesto estimado.

CU-08: Registro y Consulta de Auditoria

Cuando un microservicio necesita dejar constancia de un evento relevante, invoca POST /api/auditoria/regar en ms-auditoria con el nombre del servicio de origen y una descripcion del evento. Posteriormente, el evento puede consultarse mediante GET /api/auditoria/servicio/{nombre}, permitiendo reconstruir una bitacora centralizada del sistema.

CU-09: Reporte de un Error a GlitchTip

Si un microservicio no esta disponible (por ejemplo, ms-cliente detenido) y el BFF intenta consultarlo, se lanza una MicroserviceUnavailableException. El GlobalBffExceptionHandler la registra con log.error, retorna 503 al cliente y, si SENTRY_DSN esta configurado, el SDK de Sentry envia el evento junto con su stacktrace a GlitchTip, donde queda disponible como un issue para su analisis.

2.4 Explicacion de los Microservicios

SmartFix esta compuesto por trece componentes independientes. Cada microservicio con base de datos posee la suya propia (PostgreSQL), sin compartir tablas con otro (Database per Service). Los microservicios sin base de datos aplican una transformacion o calculo puntual sobre los datos que reciben en el request.

Servicio	Puerto	BD propia	Responsabilidad
gateway	8080	No	Unico punto de entrada publico. Reenvia todo hacia el BFF
bff-web	8083	No	Orquesta los 11 microservicios de negocio para el frontend
ms-cliente	8081	Si	Autenticacion de usuarios (JWT) y CRUD de clientes
ms-reparacion	8082	Si	CRUD de reparaciones y su ciclo de estado
ms-notificacion	8084	No	Simula el envio de notificaciones EMAIL/SMS
ms-cotizacion	8085	No	Calcula el presupuesto estimado de una reparacion
ms-garantia	8086	No	Calcula el vencimiento de la garantia
ms-conversor	8087	No	Convierte montos CLP a USD, EUR o ARS
ms-diagnostico	8088	No	Sugiere prioridad y tiempo estimado segun sintomas
ms-inventario	8089	Si	CRUD de repuestos y control de stock
ms-factura	8090	Si	Emision, consulta y anulacion de facturas
ms-encuesta	8091	Si	Encuestas de satisfaccion post-reparacion
ms-auditoria	8092	Si	Bitacora centralizada de eventos del sistema

gateway (puerto 8080)

Unico punto de entrada publico del sistema (regla de oro del proyecto). Reenvia /bff/** y el Swagger del BFF hacia bff-web usando Spring Cloud Gateway, variante WebMVC. No implementa logica de negocio ni seguridad propia.

bff-web (puerto 8083)

Backend For Frontend que orquesta llamadas a los once microservicios de negocio usando RestClient. Expone un login unificado, un dashboard combinado y un endpoint por cada microservicio bajo /bff/**. Maneja errores de forma centralizada con GlobalBffExceptionHandler, devolviendo 503 con el nombre del servicio afectado cuando este no responde.

ms-cliente (puerto 8081) y ms-reparacion (puerto 8082)

Los dos microservicios originales del proyecto (etapa C1). ms-cliente concentra la autenticacion JWT y el CRUD de clientes; ms-reparacion gestiona el ciclo de vida de las reparaciones y verifica la existencia del cliente contra ms-cliente mediante un token interno de corta duracion.

Microservicios sin base de datos: ms-notificacion, ms-cotizacion, ms-garantia, ms-conversor y ms-diagnostico

Reciben datos en el request, aplican una regla o calculo de negocio y devuelven el resultado de forma sincrona, sin necesidad de persistencia propia.

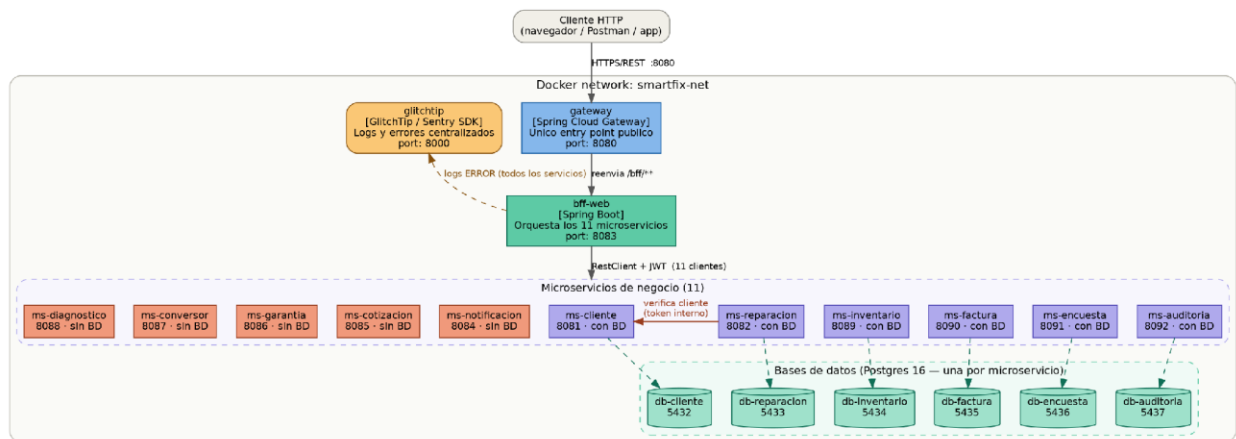
Microservicios con base de datos propia: ms-inventario, ms-factura, ms-encuesta y ms-auditoria

Cada uno cuenta con su propio PostgreSQL y sus propias migraciones Flyway. ms-inventario controla el stock de repuestos, ms-factura emite y consulta facturas, ms-encuesta mide la satisfaccion post-reparacion y ms-auditoria centraliza la bitacora de eventos del resto de los microservicios.

2.5 Diagrama C2 — Arquitectura de Contenedores

El diagrama de contenedores muestra la infraestructura completa de SmartFix: el Gateway como unico punto de entrada publico (puerto 8080), que reenvia hacia el BFF (puerto 8083); el BFF orquestando los once microservicios de negocio; cada microservicio con persistencia junto a su propia base de datos PostgreSQL; y la plataforma GlitchTip recibiendo los eventos de error de todos los componentes. Todos los contenedores operan dentro de la red Docker smartfix-net.

C4 Model — Container Diagram (C2) for SmartFix

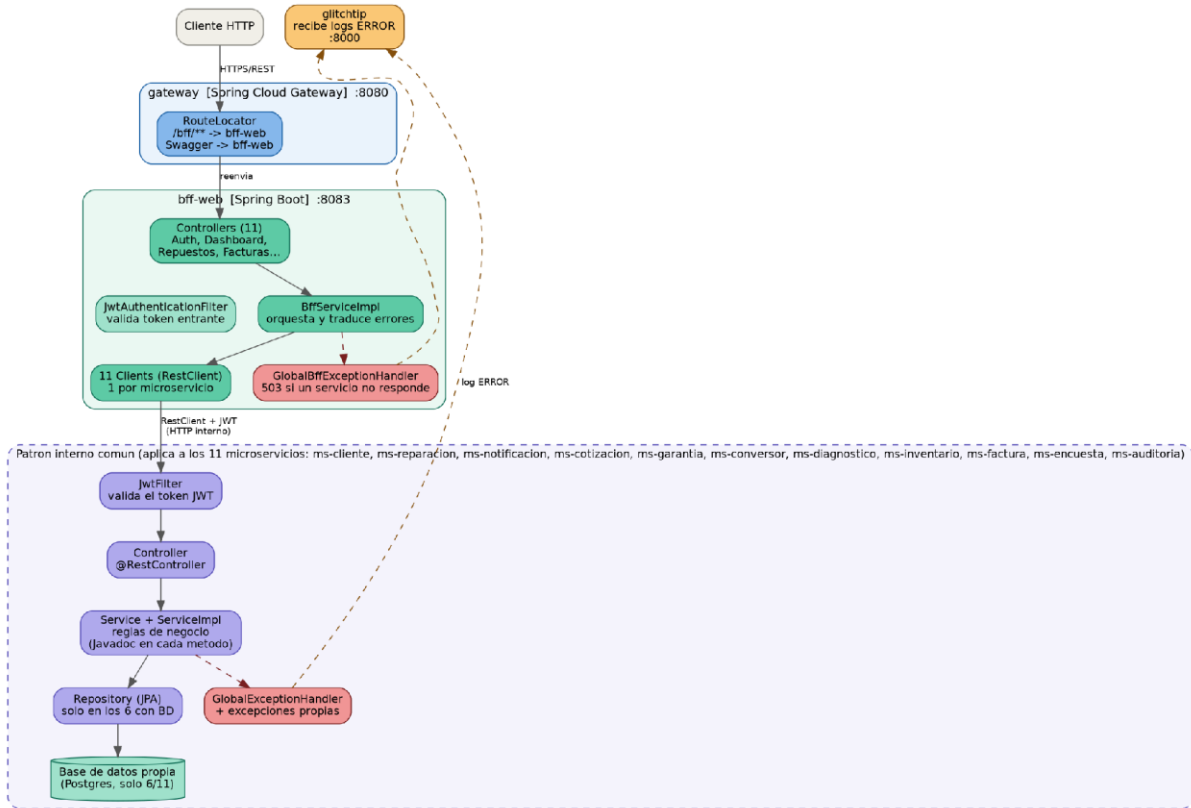


2.6 Diagrama C3 — Componentes

El diagrama de componentes expone la estructura interna del Gateway, el BFF y los nueve microservicios agregados en la ultima etapa del proyecto. En cada uno, la capa Controller recibe las peticiones y las delega a la capa Service mediante DTOs; los microservicios con persistencia acceden ademas a su Repository. Un `GlobalExceptionHandler` centraliza el manejo de errores en cada componente, y un filtro JWT valida las peticiones entrantes.

C4 Model — Component Diagram (C3) for SmartFix

Gateway, BFF y patron interno comun a los 11 microservicios



2.7 Cobertura de Pruebas

El proyecto cuenta con mas de 60 pruebas automatizadas repartidas entre los trece componentes, siguiendo la misma estrategia en todos los microservicios: pruebas unitarias de Service (con mocks de Mockito), pruebas unitarias de Controller (verificando HttpStatus y cuerpo de respuesta) y, en los microservicios con base de datos, pruebas de integracion de Repository contra una base H2 en memoria.

Modulo	Service	Controller	Repository	Total
ms-cliente	3	2	-	5
ms-reparacion	3	-	1	4
ms-notificacion	3	1	-	4
ms-cotizacion	4	1	-	5
ms-garantia	3	1	-	4
ms-conversor	2	1	-	3
ms-diagnostico	4	1	-	5
ms-inventario	5	2	2	9
ms-factura	4	2	1	7
ms-encuesta	4	2	1	7
ms-auditoria	3	2	1	6
bff-web	4	2	-	6
gateway	-	-	-	1

3. Conclusion

SmartFix demuestra la viabilidad de una arquitectura de microservicios de mayor escala para la gestión integral de reparaciones técnicas. La separación de responsabilidades entre los trece componentes del sistema Gateway, BFF y once microservicios de negocio— permite que cada servicio evolucione, se despliegue y se escale de manera independiente según la demanda de cada módulo.

La incorporación del Gateway como único punto de entrada público consolida la regla de oro del proyecto: ningún cliente externo habla directo con un microservicio de negocio, lo que simplifica la superficie de exposición del sistema y centraliza el enrutamiento. La implementación de JWT tanto para la autenticación de usuarios como para la comunicación interna entre microservicios garantiza que ningún endpoint de negocio quede expuesto sin credenciales válidas.

La incorporación de nueve microservicios adicionales —notificaciones, cotizaciones, garantías, conversión de moneda, diagnóstico, inventario, facturación, encuestas y auditoría— amplía significativamente el alcance funcional del sistema, cubriendo todo el ciclo de vida de una reparación técnica, desde el diagnóstico inicial hasta la facturación y la medición de satisfacción del cliente.

Finalmente, la integración con GlitchTip aporta observabilidad real al sistema: los errores no controlados de cualquier componente quedan registrados de forma centralizada, con su stacktrace completo, lo que reduce el tiempo de detección y diagnóstico de fallas en un entorno distribuido. Junto con las más de 60 pruebas automatizadas y la cobertura mínima exigida por JaCoCo, estos elementos constituyen una base sólida, mantenible y alineada con los estándares de la industria para sistemas de microservicios en producción.